

here's a **Product Requirements Document (PRD)** tailored to your Dev 2 scope (Node SDK + CLI + Adapters + Hosted Pay Link backend).

Product Requirements Document (PRD)

Scope: Dev 2 (Integrations & DX)

Overview

This document defines the requirements for developer experience and integration tooling for the non-custodial billing system. Dev 2 owns the CLI, SDKs, adapters, documentation, and backend support for the Hosted Pay Link.

Goals

- Provide developer-friendly tools (CLI + SDKs) for quick onboarding.
 - Build adapters (Discord, later Telegram) for instant access automation.
 - Enable Hosted Pay Link to function with invoice + status APIs (UI will be provided separately).
 - Document onboarding, webhook verification, and example integrations.
-

Out of Scope

- Core billing APIs (plans, invoices, refunds, webhooks) → owned by Dev 1.
 - Hosted Pay Link **UI design/implementation** → handled by frontend/UI team.
-

Features

1. RiskService (Lightweight DX Integration)

- Listen for `risk.flagged` and `risk.blocked` events from Dev 1.
 - Surface them in CLI logs and Node SDK responses.
 - No custom logic beyond relaying events.
-

2. Adapters

Discord Adapter (MVP)

- Event-driven bot/service.
- On `invoice.paid` or `charge.succeeded` → grant role.
- On `subscription.expired` or `refund.succeeded` → revoke role.

Config file mapping:

```
plan_pro: "@ProRole"  
plan_basic: "@BasicRole"
```

-
- Retry queue for failed Discord API calls.
- Webhook signature verification.

Telegram Adapter (Post-MVP)

- Same pattern as Discord.
 - Grant = invite user to group.
 - Revoke = kick user from group.
 - Requires wallet ↔ Telegram ID linking (handled later).
-

3. Hosted Pay Link (Backend Support)

- Expose APIs consumed by the Hosted Pay Link UI:

- `GET /invoices/{id}` → returns invoice details (merchant, plan, amount, expiry).
 - `GET /invoices/{id}/status` → returns status (`unpaid`, `processing`, `paid`, `expired`, `underpaid`).
 - Polling support or websockets for status updates.
 - No UI implemented by Dev 2.
-

4. CLI

Provide a CLI tool for merchants/devs:

- `init` → configure API key + merchant wallet.
- `plan create` → create plans.
- `invoice create` → generate invoices.
- `refund mark` → mark invoice refunded.
- `webhook test` → send mock events (`invoice.paid`, `subscription.expired`).
- `events list/retry` (optional, nice-to-have).

CLI uses mock APIs initially, real APIs when Dev 1 is ready.

5. Node SDK

Developer-friendly client for API integration:

```
import { BillingClient } from "@billing/sdk";

const client = new BillingClient({ apiKey: "sk_test_123" });

const invoice = await client.invoices.create({
  planId: "pro",
  wallet: "0x123..."
});
```

```
console.log(invoice.status); // unpaid | processing | paid
```

- Methods:
 - `plans.create(), plans.get()`
 - `invoices.create(), invoices.getStatus()`
 - `refunds.mark()`
 - `webhooks.verify(signature, payload)`
 - Promise-based, with TypeScript types.
-

6. Documentation

- Developer onboarding guide.
 - CLI install + usage examples.
 - Node SDK quickstart with code snippets.
 - Webhook verification example.
 - Example integration: Discord role automation with Pro Plan.
-

Deliverables (Dev 2)

-  CLI tool (mock + API-backed).
-  Node SDK (with quickstart examples).
-  Discord Adapter MVP.
-  Backend endpoints for Hosted Pay Link.
-  Documentation site (Docusaurus or GitBook).

Dependencies (Needs from Dev 1)

- API contracts (OpenAPI spec for `/plans`, `/invoices`, `/refunds`).
- Webhook schemas + signing method.
- Staging environment with event delivery.

Success Metrics

- New merchant can issue an invoice in <10 minutes using CLI + Hosted Pay Link.
- Node SDK works out of the box with minimal setup.
- Discord Adapter grants/revokes roles within 30s of payment events.
- Docs NPS ≥ 60 from first developers.